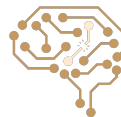


Arash
Marioriyad
8 Ordibehesht
1405

CE 417-Artificial Intelligence Sharif Univ. of Tech.

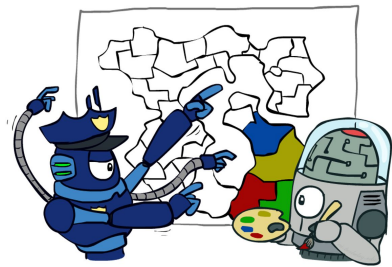
Some of Slides have been adopted from Berkeley CS188



Constraint Satisfaction Problems: Problem Formulation

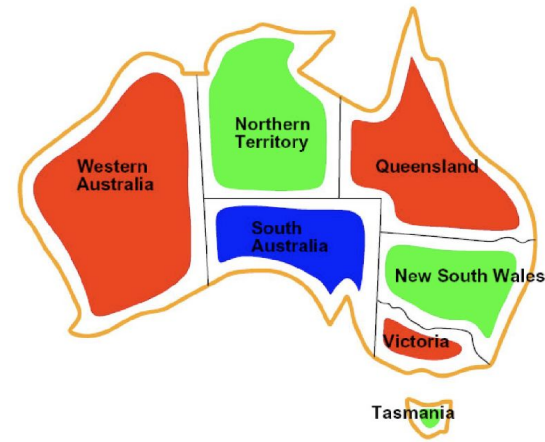
► Components of a CSP

- X is a set of **variables** $\{X_1, X_2, \dots, X_n\}$
- D is the set of **domains** $\{D_1, D_2, \dots, D_n\}$ where D_i is the domain of X_i
- C is a set of **constraints** $\{C_1, C_2, \dots, C_m\}$
 - Each constraint limits the values that variables can take (e.g., $X_1 \neq X_2$)



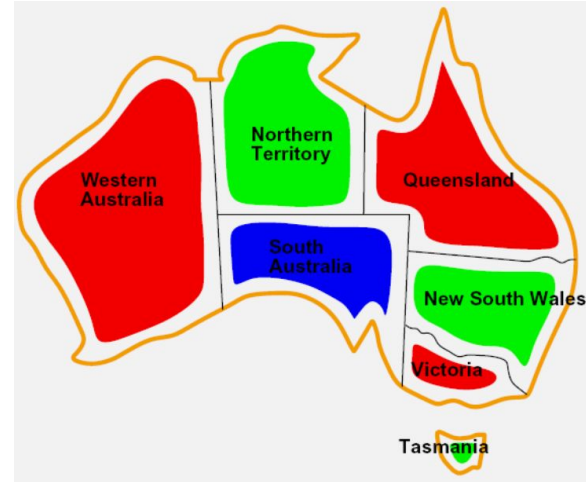
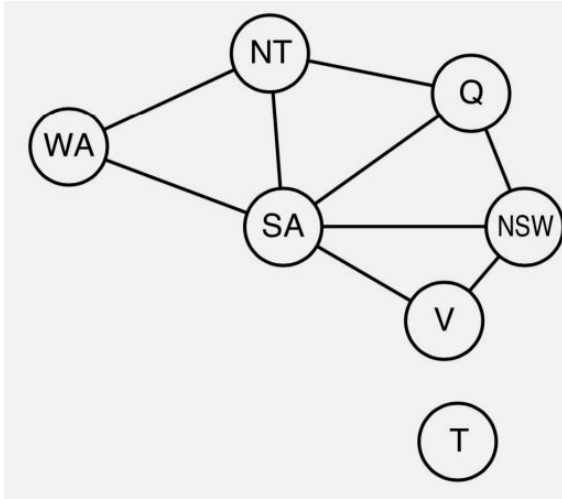
Map Coloring Example (cont.)

- **Variables:** WA, NT, Q, NSW, V, SA, T
- **Domain:** {Red, Green, Blue}
- **Constraint:** Adjacent regions must have different colors
 - **Implicit:** $WA \neq NT$
 - **Explicit:** $(WA, NT) \in \{(Red, Blue), (Red, Green), \dots\}$
- **Solutions** are assignments satisfying all constraints



Constraint graph

Constraint Graph in Binary CSPs: nodes are variables, arcs show constraints



Cryptarithmic Puzzle Example

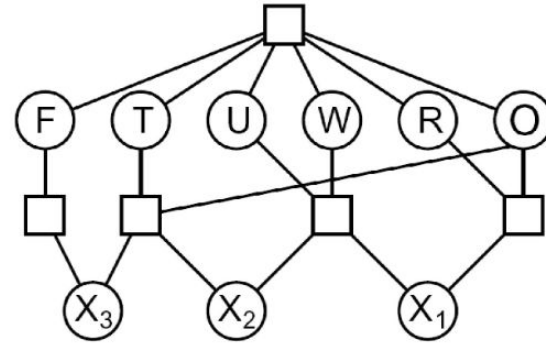
- Variables: $F T U W R O X_1 X_2 X_3$
- Domain: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- Constraints:

$\text{alldiff}(F, T, U, W, R, O)$

$$O + O = R + 10 \cdot X_1$$

...

$$\begin{array}{r} T W O \\ + T W O \\ \hline F O U R \end{array}$$



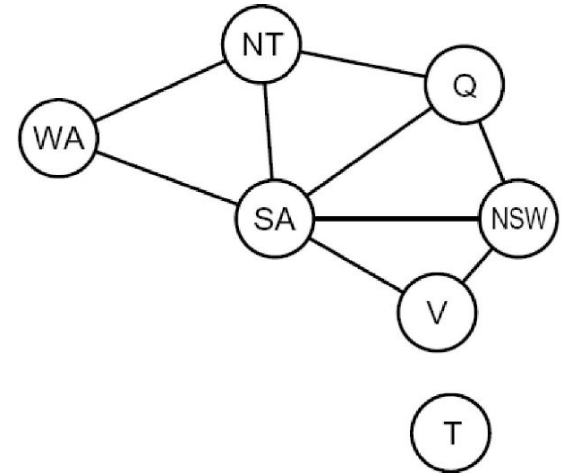
Solving SCP: Tree Search

- **States** defined by the values assigned so far (partial assignments)
- **Initial state:** the empty assignment, $\{\}$
- **Action:** assign a value to an unassigned variable
- **Goal test:** the current assignment is complete and satisfies all constraints



Solving SCP: Tree Search - BFS

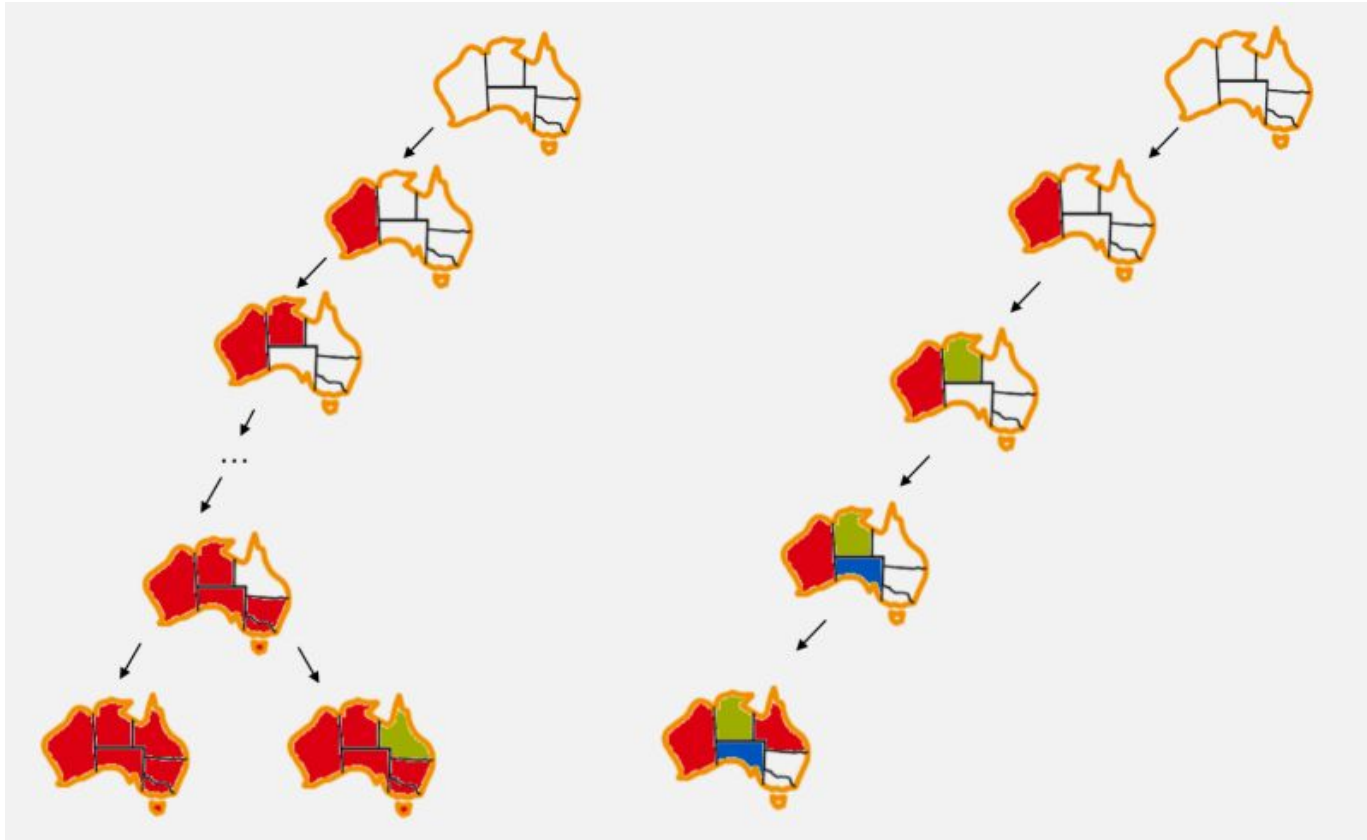
- Root: Empty assignment, $\{\}$
- Sweep over partial assignments to 1 variable:
 - $\{WA = R\}, \{WA = B\}, \{WA = G\}, \{NT = R\}, \dots$
- Then partial assignments to 2 variables:
 - $\{WA = R, NT = R\}, \{WA = R, NT = B\}, \dots$
- BFS has to go through whole tree!



All solutions are at the bottom = bad!

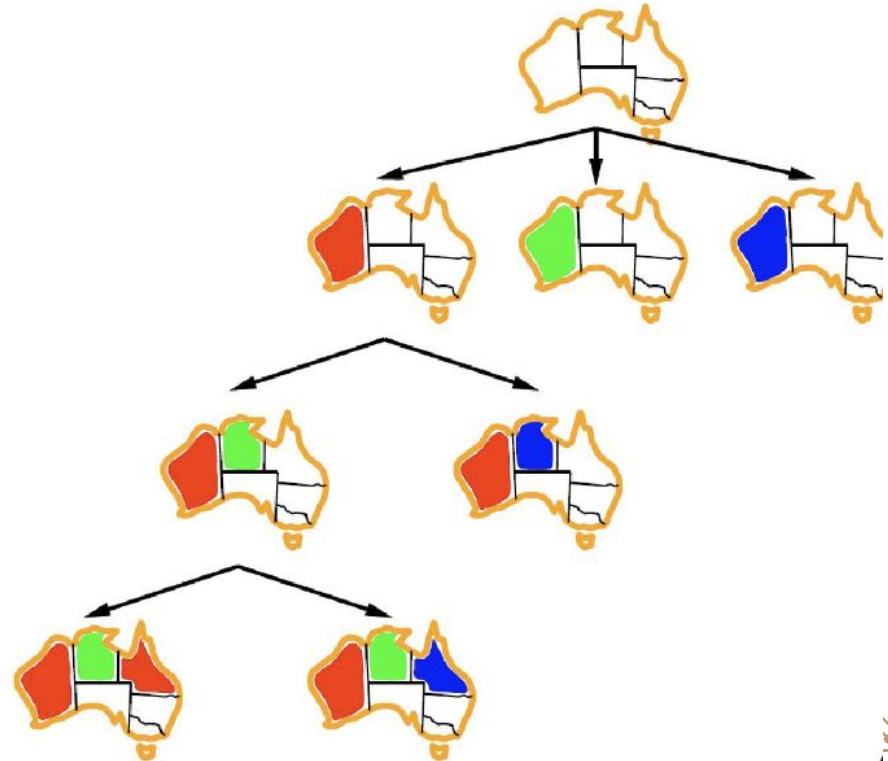


DFS vs Backtracking



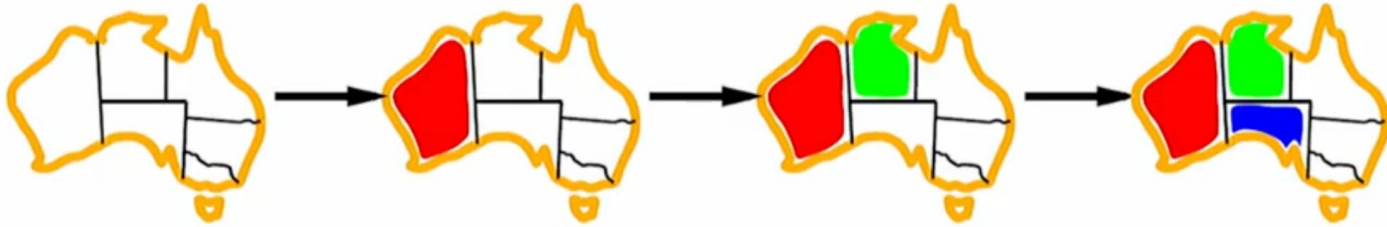
DFS vs Backtracking (cont.)

- Fixed or Dynamic Ordering Over:
 - Variables
 - Domain Values
- Intermediate Constraint Check



Ordering Heuristics: Which Variable First?

- **Variable Ordering:** Minimum remaining values (MRV)
 - Choose the variable with the **fewest legal left values** in its domain



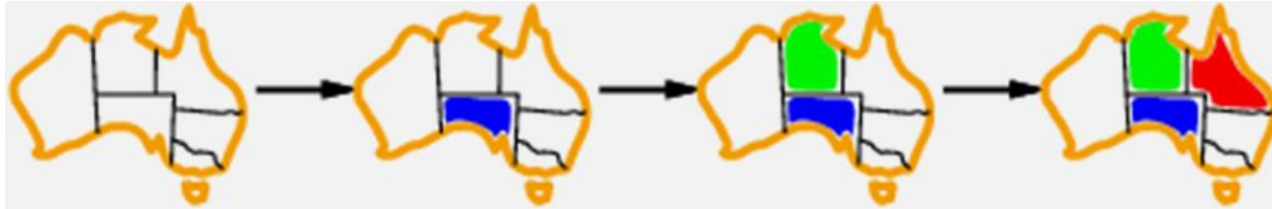
- Why min rather than max?

Also called “most constrained variable” and “Fail-fast” ordering



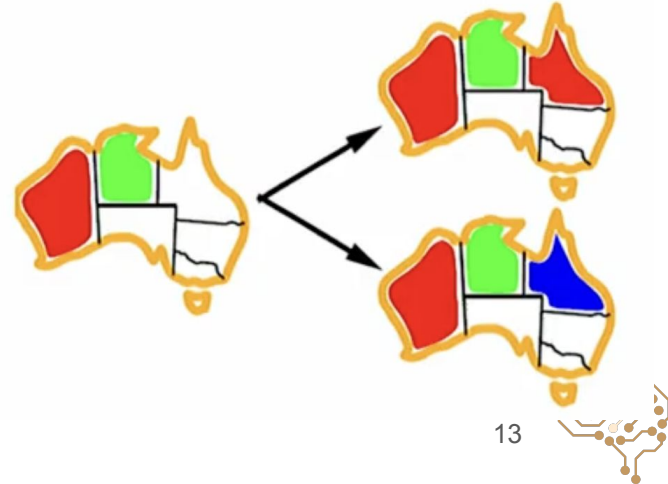
Ordering Heuristics: Which Variable First?

- **Variable Ordering:** Degree heuristic
- Choose the variable with the most constraints on remaining variables
- Tie-breaker among MRV variables



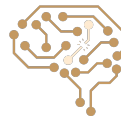
Ordering Heuristics: Which Value First?

- **Value Ordering:** Least Constraining Value
- Given a choice of variable, choose the least constraining value
- I.e., the one that rules out the fewest values in the remaining variables
- It takes extra computation to determine!
- Why the least rather than the most?



Improving Backtrack

Any Ideas?



Filtering

- **Filtering:** Pruning the domains of unassigned variables ahead of time by removing values we know will result in backtracking.
- Filtering approaches are executed **before each action selection** in backtrack search
- Two Approaches:
 - Forward Checking
 - Arc Consistency



Forward Checking

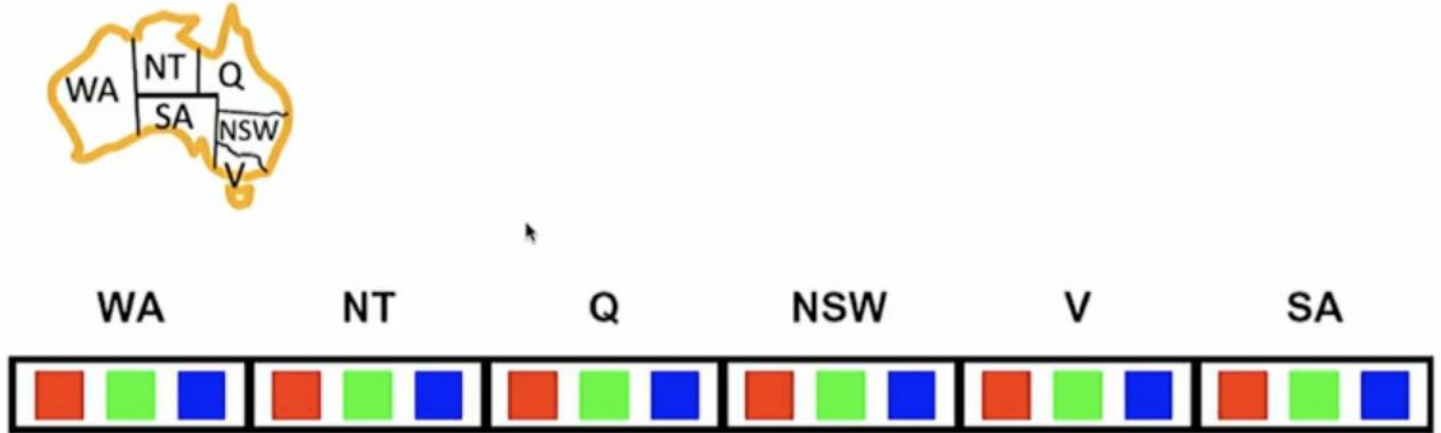
Forward Checking: whenever a value is assigned to a variable X_i , before performing the next assignment, *prunes the domains of unassigned variables that share a constraint with X_i that would violate the constraint if assigned.*



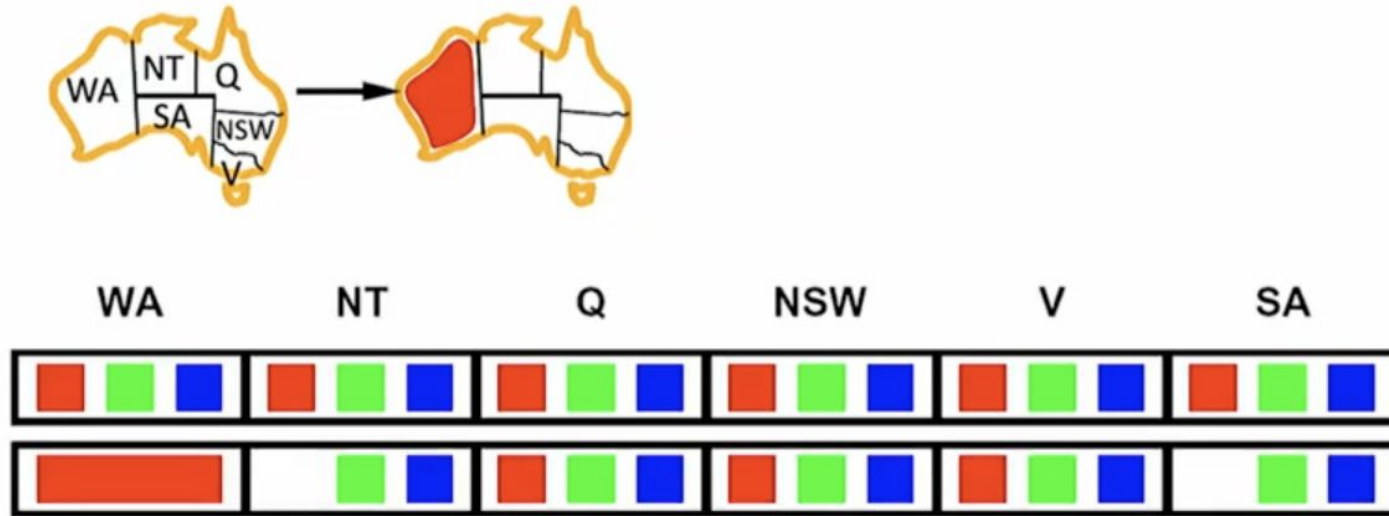
WA	NT	Q	NSW	V	SA



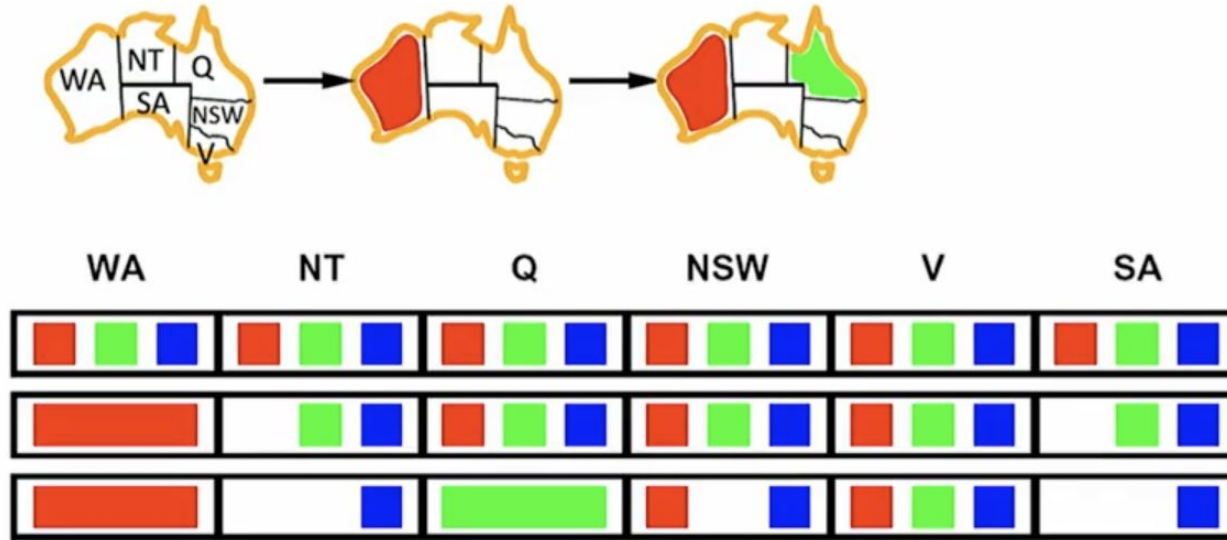
Forward Checking - Example



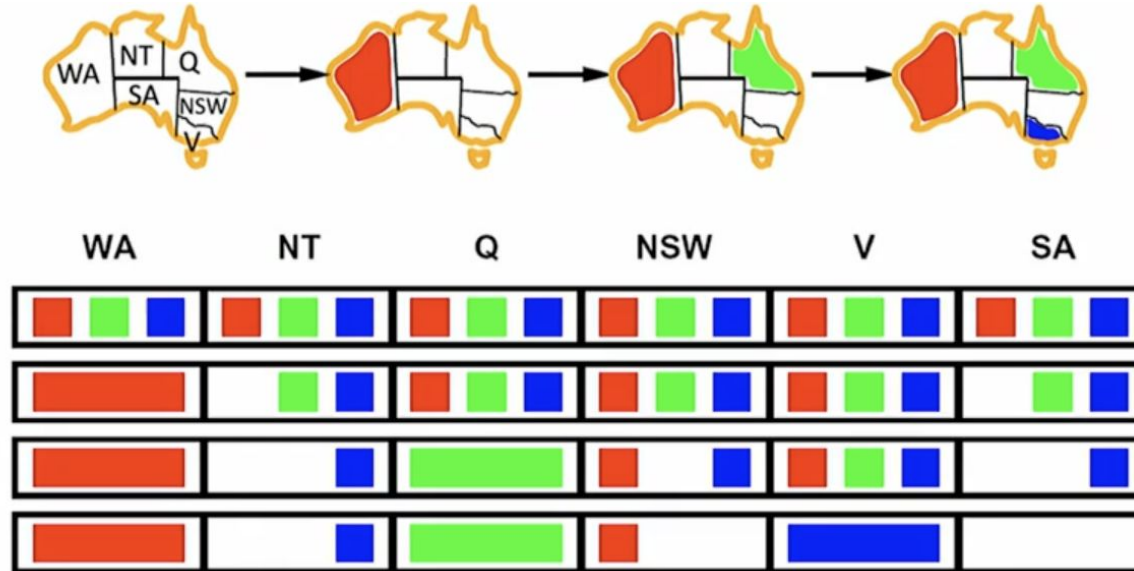
Forward Checking - Example



Forward Checking - Example

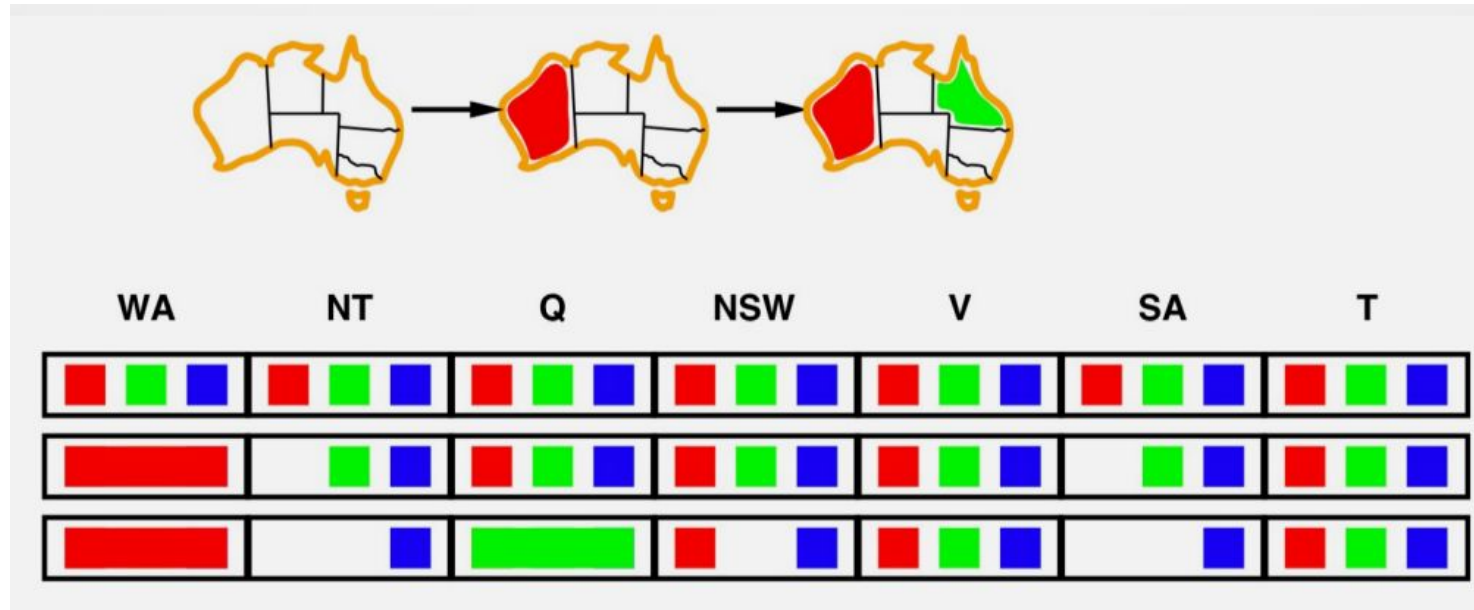
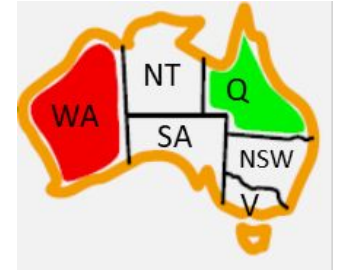


Forward Checking - Example



Forward Checking is not Enough

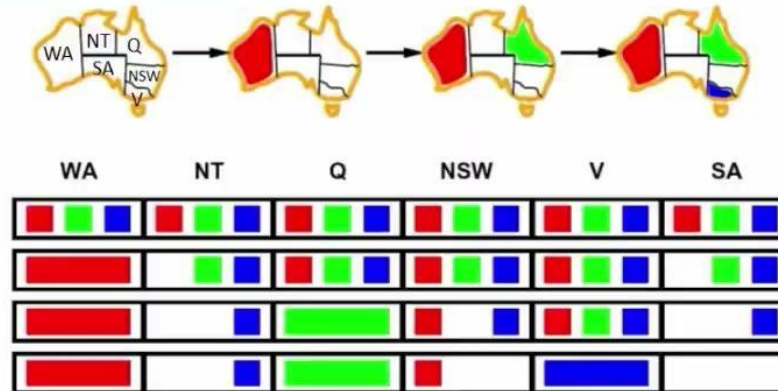
NT and SA cannot both be blue!



Forward Checking - Demo

Filtering: Forward Checking

- Filtering: Keep track of domains for unassigned variables and cross off bad options
- Forward checking: Cross off values that violate a constraint when added to the existing assignment



[demo: forward checking]



Forward Checking - Complex Graph - Demo

The screenshot displays a web browser window at the URL `beta.cs188.org/exercises/csps/forward_checking/forward_checking.html`. The main area shows a complex graph with 20 nodes, each containing three colored dots (red, blue, green). The graph is a grid-like structure with various connections. To the right of the graph are several control panels:

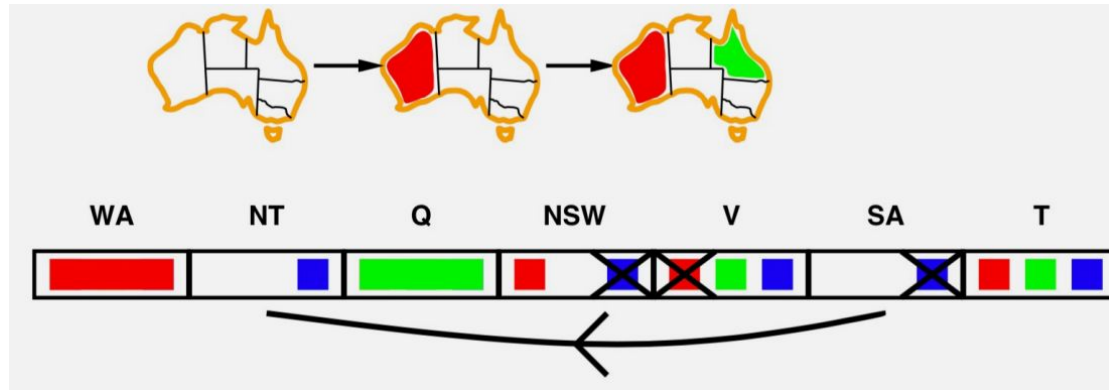
- Graph:** A dropdown menu set to "Complex".
- Algorithm:** A dropdown menu set to "Backtracking".
- Ordering:** Radio buttons for "None" (selected), "MRV", and "MRV with LCV".
- Filtering:** Radio buttons for "None", "Forward Checking" (selected), and "Arc Consistency".
- Speed:** Two input fields: "Speedup" set to "1" and "Frame Delay" set to "700".

Below the graph are buttons for "Reset", "Prev", "Pause", "Next" (with a mouse cursor over it), "Play", and "Faster". The Windows taskbar at the bottom shows the system clock as 12:14 PM on 9/4/2012.



Arc Consistency: Generalized Forward Checking

- An arc $X \rightarrow Y$ is consistent iff for every value x of X there is some allowed value y of Y .
- If X loses a value, neighbors of X need to be rechecked.
- Arc consistency detects failure earlier than forward checking. Can be run as a preprocessor or after each assignment



Arc consistency - Quiz

- X_i is arc-consistent with respect to X_j , if for every value in D_i there is a consistent value in D_j
- Example:
 - Variables: $X = \{X_1, X_2\}$
 - Domain: $\{0, 1, 2, \dots, 9\}$
 - Constraint: $X_1 = X_2^2$
- Question: Is X_1 arc-consistent w.r.t. X_2 ?
- Question: Is X_2 arc-consistent w.r.t. X_1 ?



Arc consistency - Quiz Solution

- Example:
 - Variables: $X = \{X_1, X_2\}$
 - Domain: $\{0, 1, 2, \dots, 9\}$
 - Constraint: $X_1 = X_2^2$
- Question: Is X_1 is arc-consistent w.r.t. X_2 ? No, to be arc-consistent
 $\text{Domain}(X_1) = \{0, 1, 4, 9\}$
- Question: Is X_2 is arc-consistent w.r.t. X_1 ? No, to be arc-consistent
 $\text{Domain}(X_2) = \{0, 1, 2, 3\}$
-



AC-3 Algorithm: Forcing Arc Consistency

function AC-3(*csp*) **returns** the CSP, possibly with reduced domains

inputs: *csp*, a binary CSP with variables $\{X_1, X_2, \dots, X_n\}$

local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\textit{queue})$

if REMOVE-INCONSISTENT-VALUES(X_i, X_j) **then**

for each X_k **in** NEIGHBORS[X_i] **do**

 add (X_k, X_i) to *queue*

function REMOVE-INCONSISTENT-VALUES(X_i, X_j) **returns** true iff succeeds

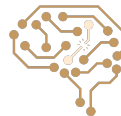
removed $\leftarrow$ false

for each x **in** DOMAIN[X_i] **do**

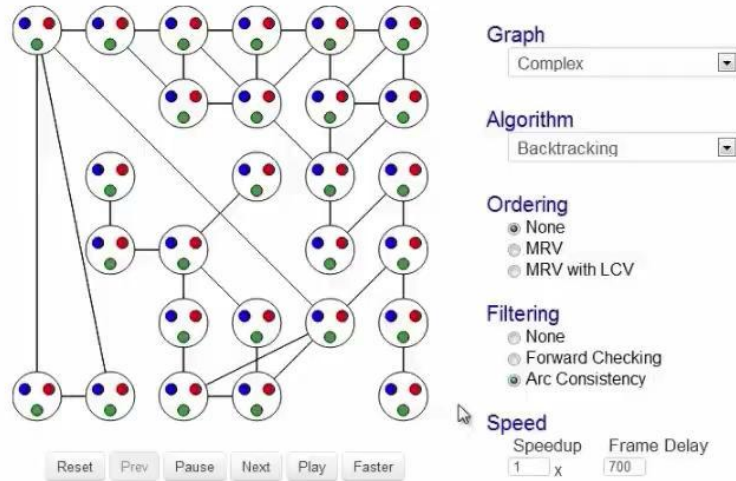
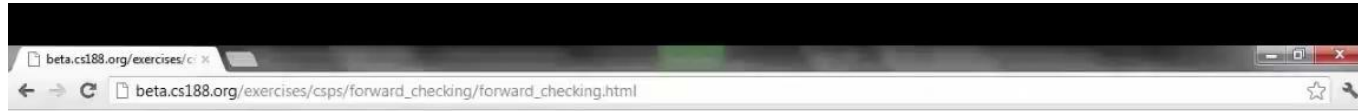
if no value y in DOMAIN[X_j] allows (x, y) to satisfy the constraint $X_i \leftrightarrow X_j$

then delete x from DOMAIN[X_i]; *removed* $\leftarrow$ true

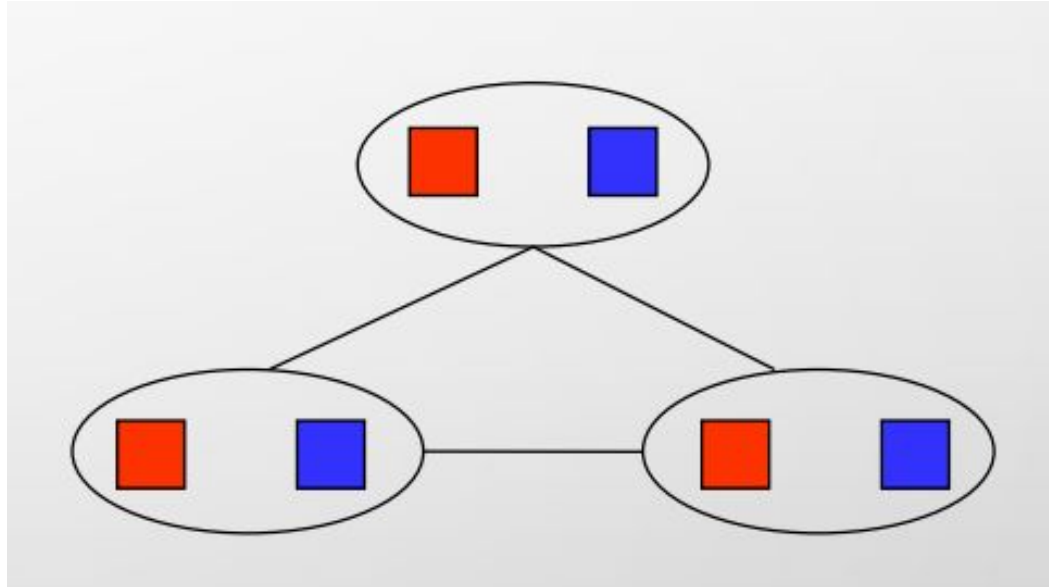
return *removed*



AC3 Algorithm (Forcing Arc Consistency) - Demo

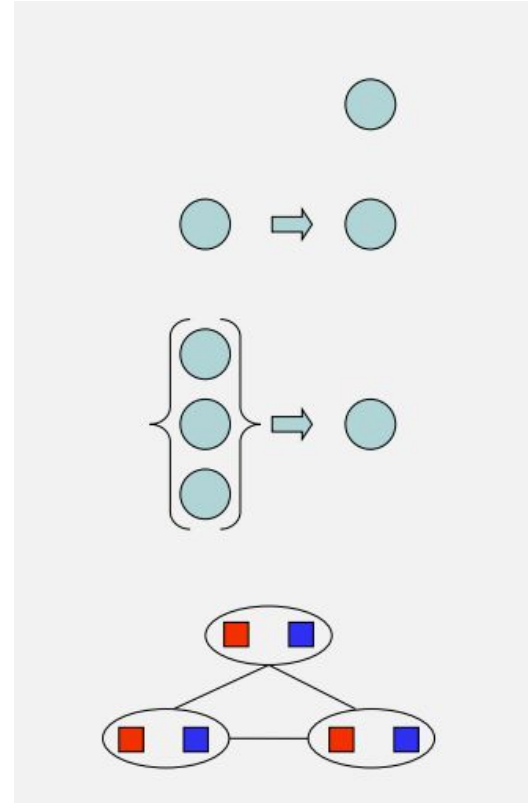


Arc Consistency Is Not Enough



Generalized Consistency

- **1-consistency (node consistency)**: each single node's domain has a value which meets that node's unary constraints
- **2-consistency (arc consistency)**: for each pair of nodes, any consistent assignment to one can be extended to the other
- **k-consistency**: for each k nodes, any consistent assignment to (k-1) node can be extended to the k-th node.

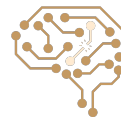
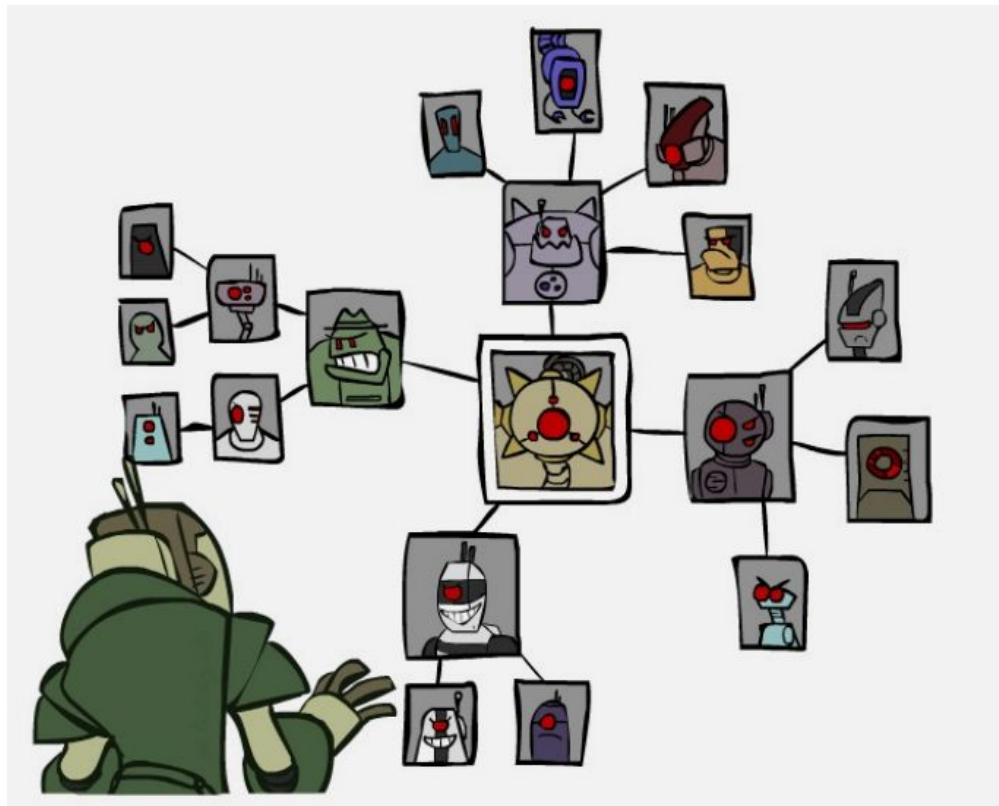


Strong K Consistency

- Strong k-consistency: 1, 2, 3, ..., k-1 consistent
- Strong n-consistency means we can solve without backtracking!
- Why?

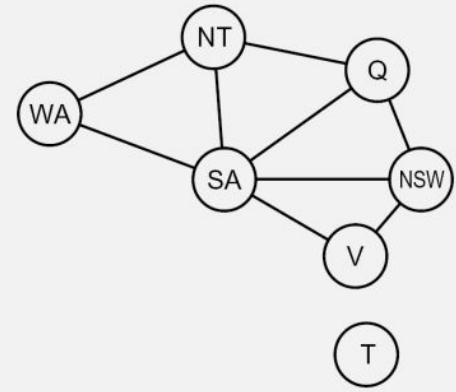


Constraint Graph Structure



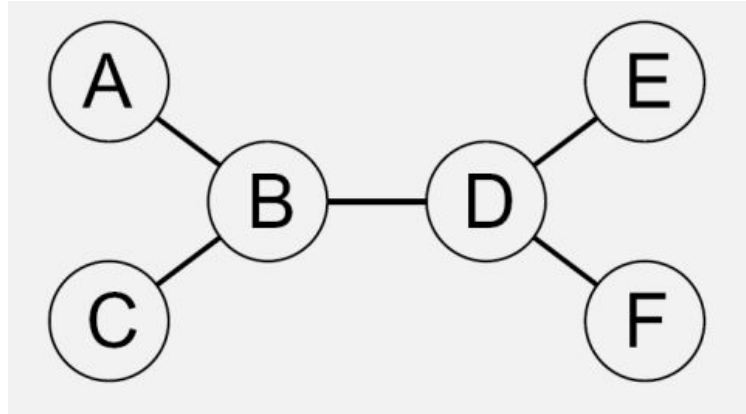
Independent Subproblems

- Extreme case: independent subproblems
 - Example: Tasmania and mainland do not interact
- Consider we can break the original problem into a number of independent subproblems
- Suppose each independent subproblem has c variables
- Worst-case solution cost is $O((n/c)(d^c))$, linear in n
- Solving the whole original problem costs $O(d^n)$ in worst-case



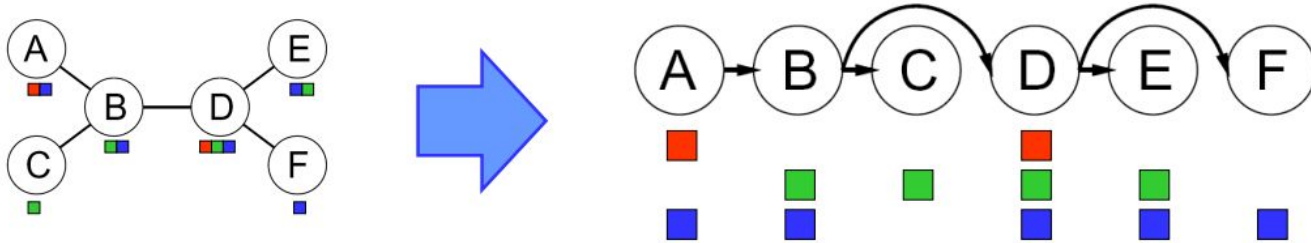
Tree Structure CSP

- **Theorem:** if the constraint graph has no loops, the CSP can be solved in $O(n.d^2)$ time
- Compare to general CSPs, where worst-case time is $O(d^n)$



Algorithm For Tree-structured CSPs: $O(n.d^2)$

- Order: Choose a root variable, order variables by topological sorting, so that parents precede children

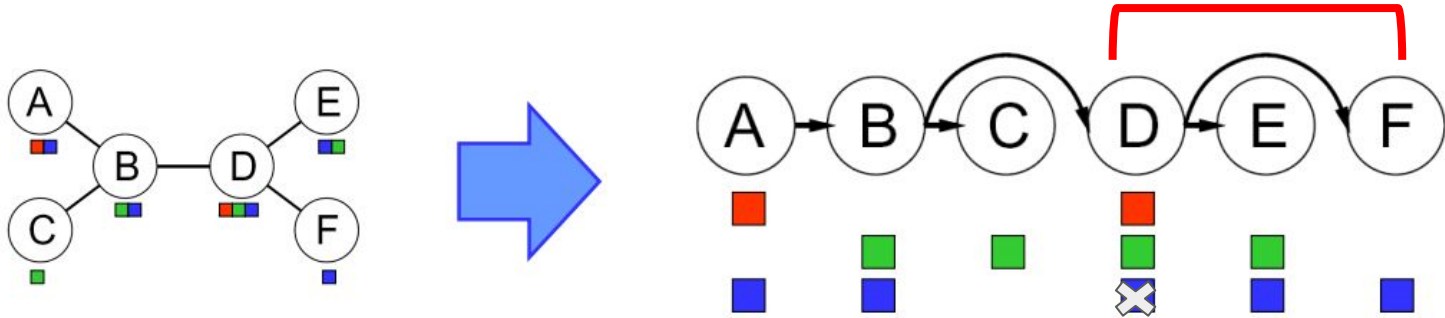


Two Phases:

- Remove backward: For $i = (n \text{ downTo } 2)$, do $\text{Remove_Inconsistent}(\text{Parent}(X_i), X_i)$
- Assign forward: For $i = (1 \text{ to } n)$, assign X_i consistently with $\text{Parent}(X_i)$



Algorithm For Tree-structured CSPs: Phase 1

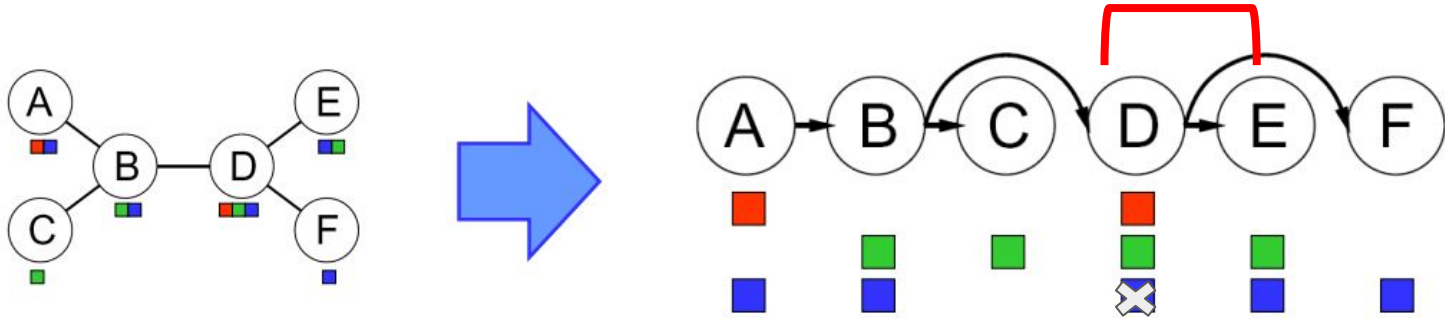


Phase 1:

- Remove backward: For $i = (n \text{ downTo } 2)$, do $\text{Remove_Inconsistent}(\text{Parent}(X_i), X_i)$



Algorithm For Tree-structured CSPs: Phase 1

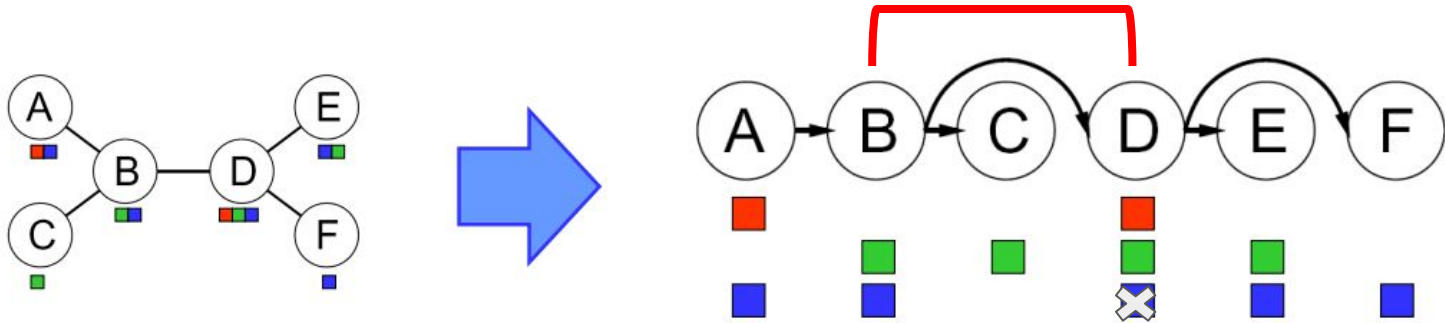


Phase 1:

- Remove backward: For $i = (n \text{ downTo } 2)$, do $\text{Remove_Inconsistent}(\text{Parent}(X_i), X_i)$



Algorithm For Tree-structured CSPs: Phase 1

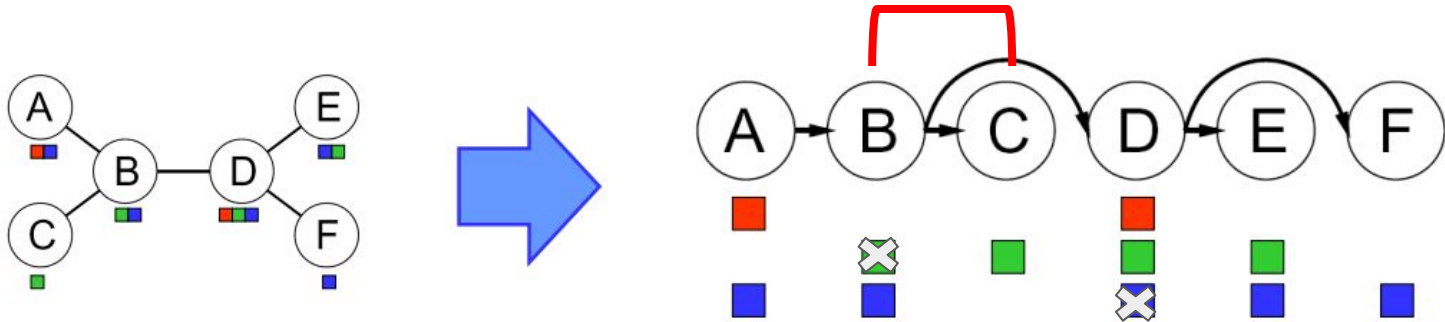


Phase 1:

- Remove backward: For $i = (n \text{ downTo } 2)$, do $\text{Remove_Inconsistent}(\text{Parent}(X_i), X_i)$



Algorithm For Tree-structured CSPs: Phase 1

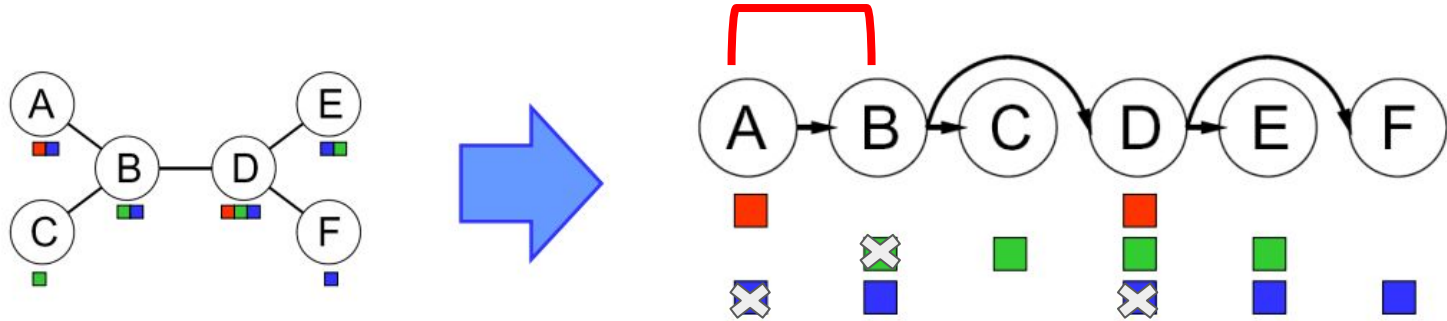


Phase 1:

- Remove backward: For $i = (n \text{ downTo } 2)$, do $\text{Remove_Inconsistent}(\text{Parent}(X_i), X_i)$



Algorithm For Tree-structured CSPs: Phase 1

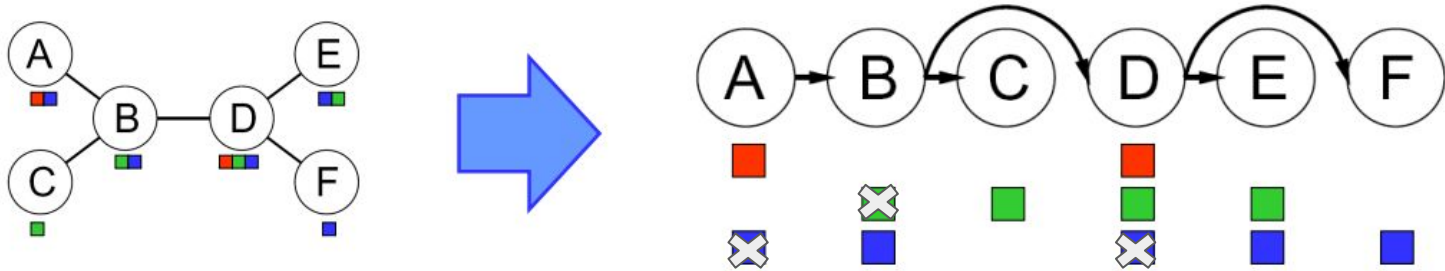


Phase 1:

- Remove backward: For $i = (n \text{ downTo } 2)$, do $\text{Remove_Inconsistent}(\text{Parent}(X_i), X_i)$



Algorithm For Tree-structured CSPs: Phase 1

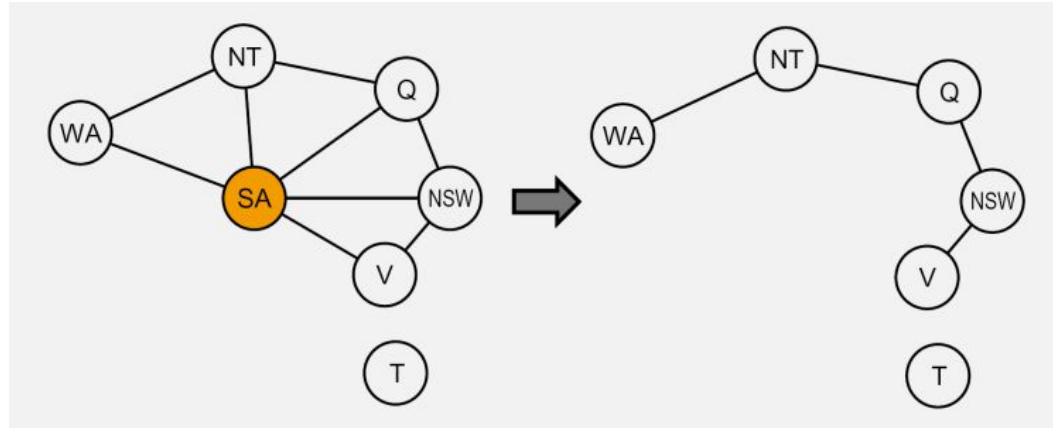


Phase 2 (Assign forward):

- A=Red, B=Blue, C=Green, D=Red, E=Green, F=Blue



Nearly tree-structured CSPs



- Instantiate a variable
- Such that the remaining constraint graph is a tree
- Prune the other variable domains



Next Session: Adversarial Multi-Agent Search

